40. RBAC Policy DSL Compiler with Static Security & Privilege Escalation Detection

Course Name: Compiler Design

Course Code: CS1202

Name: Pinikeshi Meghana

Roll No: 24CSB0A54

Sec: CSE-A

Document Details: Week 8 Deliverables

Week 8 – Role Conflict & Redundancy Detection

In Week 8, the RBAC DSL Compiler was enhanced with a dedicated Security Analysis Phase.
This phase performs static validation of role-based access control (RBAC) policies to detect security design flaws before deployment.

The implementation focuses on:

- Detection of mutually exclusive role assignments
- Identification of redundant permissions introduced via inheritance
- Enforcement of least-privilege principles

This improves the correctness, clarity, and security of RBAC policies.

1.Objectives

The objectives of Week 8 are:

- Detect role conflicts using mutex constraints
- Identify redundant permissions caused by role inheritance
- Enforce least-privilege rules
- Generate an automated security analysis report

2. Security Analysis Phase

A new phase was added after Semantic Analysis:

Lexical Analysis
↓
Parsing & Symbol Table Construction
↓
Semantic Analysis
↓
Security Analysis (Week 8)

The Security Analysis operates on the fully constructed Symbol Table and checks for policy-level violations.

3. Implemented Features

3.1 Role Conflict Detection (Mutex Rule)

The DSL supports mutually exclusive roles using the syntax:

mutex RoleA RoleB

If a user is assigned both roles in a mutex pair, a violation is reported.

Example Input:

```
role Admin {
   permissions = [read, write]
}

role Auditor {
   permissions = [read]
}

user Alice {
   roles = [Admin, Auditor]
}

mutex Admin Auditor
```

Output:

[ROLE CONFLICT] User 'Alice' has mutually exclusive roles 'Admin' and 'Auditor'

This prevents conflicting privilege combinations.

3.2 Redundant Permission Detection

When a role inherits from a parent role, it automatically receives all parent permissions.

If the child role explicitly redefines a permission already inherited, it is considered redundant.

Example Input:

```
role Base {
   permissions = [read]
}

role Manager extends Base {
   permissions = [read, write]
}
```

Output:

[REDUNDANCY] Role 'Manager' redefines inherited permission 'read'

This improves policy clarity and avoids unnecessary duplication.

3.3 Least Privilege Enforcement

The system enforces least-privilege using static checks:

- Roles with no permissions
- Roles with no effective permissions
- Users assigned undefined roles

Example Input:

```
role Guest {
}

user Bob {
   roles = [Guest]
}
```

Output:

[LEAST PRIVILEGE] Role 'Guest' has no effective permissions

This ensures that all roles provide meaningful and minimal access.

4. Security Rules Implemented

Rule	Description
Mutex Role Rule	Prevents assignment of mutually exclusive roles to a user
Redundancy Rule	Detects redefinition of inherited permissions
Least Privilege Rule	Flags roles with no effective permissions
Inheritance Validation	Ensures no circular inheritance in role hierarchy

5. Security Report Generation

The system automatically generates:

week8_security_report.txt

This file contains all detected security violations in a structured format.

If no violations are found, the file is created as empty.

6. Results

The Week 8 implementation successfully:

- Detects conflicting role assignments
- Identifies redundant permissions
- Enforces least-privilege constraints
- Generates automated security reports

The modular design allows future extension of additional RBAC security rules.